

AI Homework 4

112550184 陳璽安

1 Introduction

In this assignment, I implement three Python files: `main.py`, `Agent.py`, `BanditEnv.py`.

A game environment for k-armed bandit problem is implemented in `BanditEnv.py`. Which includes stationary environment and non-stationary environment.

In `Agent.py`, the agent is implemented to decide the next step in the k-armed Bandit problem with ϵ -greedy exploration. We will record the actions and rewards during the experiment and plot the curves to visualize the affects of these agents in `main.py`.

2 Implementation detail

2.1 Game Environment Implementation

The game environment is implemented in `BanditEnv.py`. This class provides the foundation for both stationary and non-stationary k-armed bandit problems.

It includes: `__init__()` to initialize the k-armed bandit environment, `reset()` to reset the environment, `step(action)` to take an action in the process and return the reward, and `export_history()` to return the action history and reward history. Below are the implementation for `BanditEnv.py`

`__init__()` takes parameters `k` and `stationary`(default to be `True`). Initialize `self.k` and `self.stationary` with the parameters and set the means of the reward for each arms to be a random number(generated by `np.random.normal(0,1,k)` let the reward distribution to be a Gaussian distribution with variance 1)

And set `action_history` and `reward_history` to be two empty lists.

```
1 class BanditEnv():
2     def __init__(self, k, stationary = True):
3         self.k = k
4         self.stationary = stationary
5         self.arm_means = np.random.normal(0,1,k)
6         self.action_history = []
7         self.reward_history = []
```

`reset()` function regenerate the reward means for each arms by `np.random.normal(0,1,self.k)` function and reset the `action_history` and `reward_history` to be empty.

```
1 def reset(self):
2     self.arm_means = np.random.normal(0,1,self.k)
3     self.action_history = []
4     self.reward_history = []
```

`step(action)` takes parameter `action` and generate reward by normal distribution with `mean=self.arm_means[action]`

Append the action and reward the action history and reward history.

If it's a non-stationary environment, we need to add the true mean of rewards to a random number which has a distribution of normal distribution with mean 0 and standard deviation 0.01.

Finally, return the reward.

```
1 def step(self, action):
2     reward = np.random.normal(self.arm_means[action],1)
3     self.action_history.append(action)
4     self.reward_history.append(reward)
5     if self.stationary == False:
6         for i in range(self.k):
7             self.arm_means[i]+=np.random.normal(0,0.01)
8     return reward
```

The `export_history()` function returns two lists `action_history` and `reward_history`

```
1 def export_history(self):
2     return self.action_history, self.reward_history
```

2.2 Agent Implementation

The Agent decides actions in k-armed bandit problem is implemented in `Agent.py`. It includes `__init__()` function to initialize the agent; `select_action()` choose the next action based on ϵ value and current Q-value(expected mean value for each arms); `reset()` function to reset the agent and `update_q(action, reward)` function to update the Q-value after toke each action.

Below are the code explanation:

Initialize `k`(arms number), `epsilon`, `alpha` value with parameters. And initialize two lists `q_val` and `action_count` to be `k` zeros. The `q_val` list will store the expected mean value for each arms, and `action_count` list record how many times we choose the arm.

```
1 class Agent():
2     def __init__(self,k, epsilon,alpha=None):
3         self.k =k
4         self.epsilon=epsilon
5         self.q_val = [0]*k
6         self.action_count = [0]*k
7         self.alpha = alpha
```

Generate a random number to implement ϵ -greedy exploration: If the random number is less than ϵ , do exploration (choose a random arm to be next action). Otherwise, do exploitation (choose the arm has the highest expected mean so far).

```
1 def select_action(self):
2     if random.random()<self.epsilon:
3         action = random.randint(0,self.k-1)
4     else:
5         action = np.argmax(self.q_val)
```

```
6         return action
```

Reset the agent: Make action counts and Q-value(expected mean) of each arms to be zero.

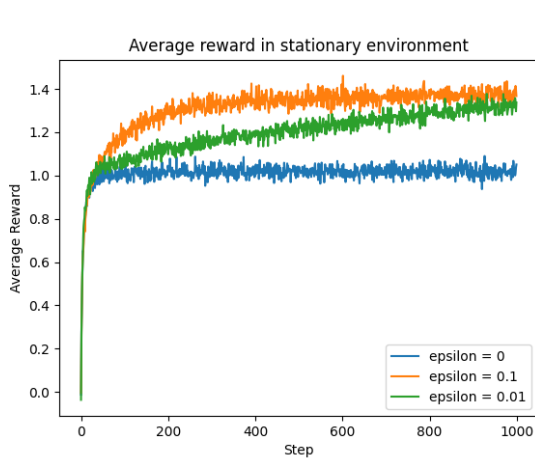
```
1     def reset(self):
2         self.action_count=[0]*self.k
3         self.q_val=[0]*self.k
```

After taking an action, we need to update Q-value. When alpha is None, update Q-value as the running average of all rewards received so far for this action. If alpha is provided, update Q-value with constant step-size update method: $Q(a) \leftarrow Q(a) + \alpha \cdot (R - Q(a))$

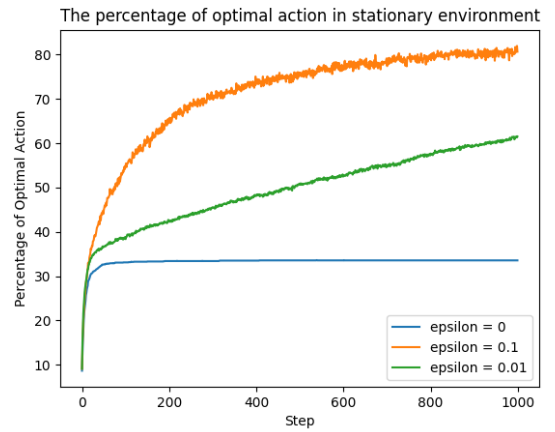
```
1     def update_q(self, action, reward):
2         if self.alpha == None:
3             self.action_count[action]+=1
4             self.q_val[action]=(self.q_val[action]*(self.action_count[action]-1)+reward)/self
5             ↪ .action_count[action]
6
7         else:
8             self.q_val[action] = self.q_val[action]+self.alpha*(reward-self.q_val[action])
```

3 Experiment Results

3.1 In Stationary Environment



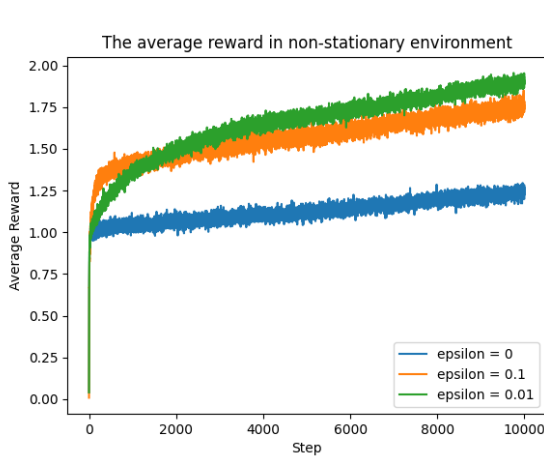
(a) Average rewards in stationary environment



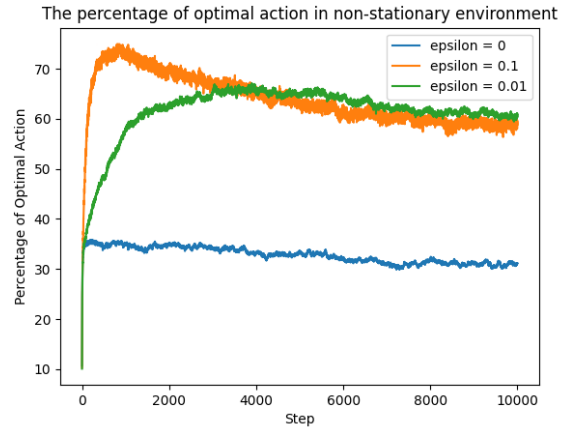
(b) Percentage of optimal action in stationary environment

In the figure of average rewards, $\epsilon=0.1$ performs best (1.4), followed by $\epsilon=0.01$ (1.3), while $\epsilon=0$ performs worst (1.0). As for optimal action selection, $\epsilon = 0.1$ reaches 80%, $\epsilon = 0.01$ reaches 60%, $\epsilon = 0$ only 35%. It shows that appropriate exploration finds truly optimal options, pure greed gets stuck in suboptimal solutions

3.2 In Non-stationary Environment



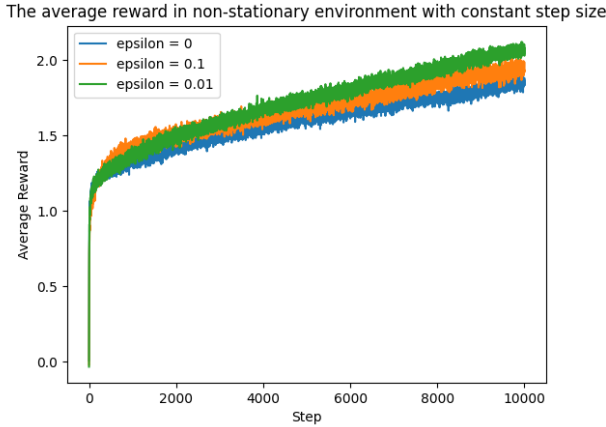
(a) Average rewards in non-stationary environment



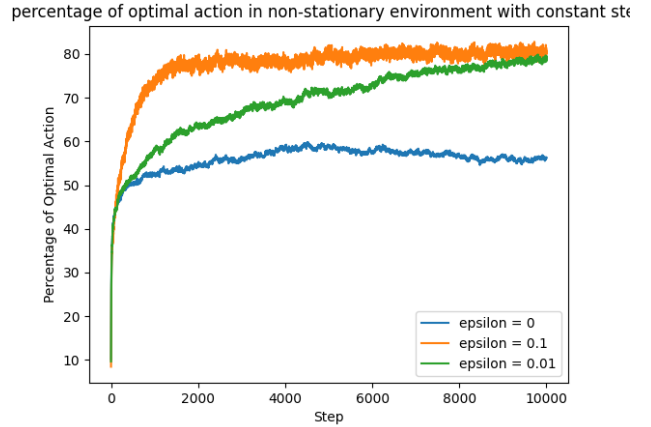
(b) Percentage of optimal action in non-stationary environment

Average rewards for $\epsilon = 0.01$ and $\epsilon = 0.1$ perform similarly (1.8), much better than $\epsilon = 0$ (1.25). As for optimal action selection, all strategies gradually decline over time, showing difficulty tracking optimal actions as environment changes.

3.3 In non-stationary environment with constant step-size update



(a) Average rewards in non-stationary environment with constant step-size



(b) Percentage of optimal action in non-stationary environment with constant step-size

As you can see from the figure of average rewards, all three strategies perform better than with standard updates, $\epsilon = 0.01$ performs slightly better than $\epsilon = 0.1$. In the figure of optimal selection, $\epsilon = 0.1$ maintains stable 80%, $\epsilon = 0.01$ gradually approaches 80%, $\epsilon = 0$ improves to 55-60%. From the picture above, agents with constant step-size update method perform much better than agents updating expected mean as running average.

3.4 Summary

1. Moderate exploration always performs better than pure exploitation.
2. In non-stationary environments, continuous exploration and appropriate update methods are crucial.

3. Constant step-size updates excel in non-stationary environments, effectively tracking changing reward distributions.

4 Discussion

Regarding the value of ϵ . In both stationary and non-stationary environments, purely greedy approaches ($\epsilon = 0$) always performs worst compared to strategies that incorporate exploration. Besides, best ϵ value may differ depends on the environment. In the stationary environment, $\epsilon = 0.1$ clearly outperformed $\epsilon = 0.01$. However, in the non-stationary environment with constant step-size updates, $\epsilon = 0.01$ slightly outperformed $\epsilon = 0.1$.

Besides, the results above clearly shows constant step-size outperforms sample-average method (update the Q-value as running average) in non-stationary environments. It's because sample-average treats all historical rewards equally, while constant step-size updates prioritize recent rewards, allowing the agent to adapt more quickly to changing reward distributions.

Although sample-average method is more straight forward, it's crucial to use the appropriate updating method in non-stationary environment. And we should choose the optimal value of ϵ based on environment conditions. This assignment deepened my understanding of the exploration-exploitation trade-off and how different update methods are suited to different environmental conditions.